

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	M.MANASA	Reg. No.:	192425189
Section / Batch:	2024 AIML	Date:	19/8/2026

Code of Conduct

I M.MANASA(192425189) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: m.manasa

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} | \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	7	7	6	6	5	32	93
2	6	5	6	5	5	32	
3	7	7	5	6	6	27	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Case Study 1: E-Commerce Smart Search and Product Prediction System

Given Data

Corpus: “machine learning improves business, machine learning enables automation, machine learning drives innovation.”

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$, $C(\text{learning enables}) = 1$, $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$
- Interpolation weights: λ_1 (trigram) = 0.5, λ_2 (bigram) = 0.3, λ_3 (unigram) = 0.2
- Prediction distribution after “learning”: $P(\text{improves}) = P(\text{enables}) = P(\text{drives}) = 0.33$

Q1. Compute the MLE Bigram Probability $P(\text{learning} \mid \text{machine})$ and interpret its effect on next-word prediction.

$$P(\text{learning} \mid \text{machine}) = C(\text{machine, learning}) / C(\text{machine}) = 3 / 3 = 1.0$$

Interpretation: A bigram probability of 1.0 means that, within this training corpus, every single occurrence of the word “machine” is immediately followed by “learning” — there is no competing continuation. For the e-commerce autocomplete system this is the strongest possible signal: as soon as a customer types “machine”, the search box can confidently suggest “learning” as the next token with no ambiguity. In practice this kind of certainty is rare and usually a symptom of a very small training corpus; a production system would still smooth this estimate so that novel continuations (e.g. “machine washable”) are not assigned zero probability outright.

Q2. Apply the Backoff Model to estimate $P(\text{transforms})$ for the unseen query “machine learning transforms” and explain why backoff is useful.

Step 1 (Trigram): $C(\text{machine learning transforms}) = 0$ in the corpus, so the trigram estimate $P(\text{transforms} \mid \text{machine, learning})$ is undefined/zero. The model backs off.

Step 2 (Bigram): $C(\text{learning transforms}) = 0$ as well (“transforms” never follows “learning” in the corpus), so the bigram estimate $P(\text{transforms} \mid \text{learning})$ is also zero. The model backs off again.

Step 3 (Unigram): $C(\text{transforms}) = 0$ across the whole corpus (the word never appears), so even the raw unigram estimate is zero. A pure backoff without smoothing would therefore assign $P(\text{transforms}) = 0$, which is the classic “zero-probability problem”.

$$\text{Katz-style backoff: } P_{\text{bo}}(w \mid \text{context}) = P(w \mid \text{context}) \text{ if seen, else } \alpha(\text{context}) \times P_{\text{bo}}(w \mid \text{shorter context})$$

Because “transforms” is unseen at every order, a smoothing method (e.g. add- α / Good-Turing discounting) reserves a small held-out probability mass from seen events and redistributes it to unseen words, giving “transforms” a small non-zero probability instead of exactly 0.

Why backoff is useful: e-commerce search queries constantly contain novel word combinations that a fixed-size training corpus could never fully cover. Backoff lets the system gracefully fall back from the most specific, most informative estimate (trigram) to progressively more general and more reliably-estimated ones (bigram $\rightarrow$

unigram) rather than failing outright, so autocomplete keeps returning a ranked, non-zero suggestion list even for queries it has never seen before.

Q3. Using Deleted Interpolation, calculate $P(\text{"machine learning improves"})$ and explain the benefit of combining trigram, bigram and unigram probabilities.

Component estimates (total corpus length = 12 words):

$$P_3(\text{improves} / \text{machine, learning}) = C(\text{machine, learning, improves}) / C(\text{machine, learning}) = 1 / 3 = 0.333$$

$$P_2(\text{improves} / \text{learning}) = C(\text{learning, improves}) / C(\text{learning}) = 1 / 3 = 0.333$$

$$P_1(\text{improves}) = C(\text{improves}) / N = 1 / 12 = 0.083$$

$$\begin{aligned} P_{\text{interp}}(\text{improves} / \text{machine, learning}) &= \lambda_1 P_3 + \lambda_2 P_2 + \lambda_3 P_1 \\ &= (0.5 \times 0.333) + (0.3 \times 0.333) + (0.2 \times 0.083) = 0.1667 + 0.1000 + 0.0167 \approx 0.283 \end{aligned}$$

Interpreting the sequence probability: $P(\text{machine learning improves}) \approx P(\text{machine}) \times P(\text{learning} | \text{machine}) \times P_{\text{interp}}(\text{improves} | \text{machine, learning}) \approx (3/12) \times 1.0 \times 0.283 \approx 0.071$.

Benefit of interpolation: The trigram estimate alone is fragile because it is based on a single observed occurrence, so its reliability is low. Deleted interpolation blends it with the more frequently observed bigram and the very robust unigram estimate, weighting the sparser but more context-specific trigram most heavily ($\lambda_1 = 0.5$) while still letting the stable, data-rich lower-order estimates cushion the prediction. This makes the model far more robust to data sparsity than relying on any single N-gram order alone.

Q4. Calculate the entropy of the prediction distribution $P(\text{improves})=P(\text{enables})=P(\text{drives})=0.33$ and interpret it.

$$H(X) = -\sum p(x) \log_2 p(x) = -3 \times (0.33 \times \log_2 0.33)$$

$$\log_2(0.33) \approx -1.599 \Rightarrow H(X) \approx -3 \times (0.33 \times -1.599) \approx 1.58 \text{ bits}$$

Interpretation: The maximum possible entropy for a 3-way choice is $\log_2(3) \approx 1.585$ bits, which occurs exactly when all outcomes are equally likely. Our computed value (≈ 1.58 bits) is essentially at this maximum, meaning the model has no useful preference among “improves”, “enables” and “drives” — the search system is at maximum uncertainty about the next word. This signals that, at this point in the query, the language model needs more context (or a richer training corpus) before it can rank one suggestion above the others with confidence.

Q5. Python program: build Unigram/Bigram/Trigram models, compute MLE, Backoff, Deleted Interpolation and Entropy, and display all intermediate results.

```
import nltk
import numpy as np
import math
from nltk import word_tokenize
from nltk.util import ngrams
from collections import Counter

nltk.download('punkt', quiet=True)

# ---- 1. Corpus & tokenisation ----
corpus = ("machine learning improves business, machine learning enables automation, "
          "machine learning drives innovation.")
tokens = [w.lower() for w in word_tokenize(corpus) if w.isalpha()]
```

```

# ---- 2. Build Unigram / Bigram / Trigram frequency models ----
unigrams = Counter(ngrams(tokens, 1))
bigrams = Counter(ngrams(tokens, 2))
trigrams = Counter(ngrams(tokens, 3))
N = len(tokens)

def mle_bigram(w2, w1):
    """P(w2 | w1) via MLE"""
    return bigrams[(w1, w2)] / unigrams[(w1,)] if unigrams[(w1,)] else 0.0

def backoff_prob(word, context, alpha=0.4):
    """Simple Katz-style backoff for a trigram context (w1, w2) -> word"""
    w1, w2 = context
    if trigrams[(w1, w2, word)] > 0:
        return trigrams[(w1, w2, word)] / bigrams[(w1, w2)]
    elif bigrams[(w2, word)] > 0:
        return alpha * (bigrams[(w2, word)] / unigrams[(w2,)])
    elif unigrams[(word,)] > 0:
        return alpha * alpha * (unigrams[(word,)] / N)
    else:
        return alpha * alpha * (1 / (N + 1)) # smoothed floor for unseen word

def deleted_interpolation(word, w1, w2, lambdas=(0.5, 0.3, 0.2)):
    l1, l2, l3 = lambdas
    p3 = trigrams[(w1, w2, word)] / bigrams[(w1, w2)] if bigrams[(w1, w2)] else 0
    p2 = bigrams[(w2, word)] / unigrams[(w2,)] if unigrams[(w2,)] else 0
    p1 = unigrams[(word,)] / N
    return l1 * p3 + l2 * p2 + l3 * p1

def entropy(dist):
    return -sum(p * math.log2(p) for p in dist if p > 0)

# ---- 3. Run all computations ----
print('MLE P(learning|machine)      :', round(mle_bigram('learning', 'machine'), 3))
print('Backoff P(transforms|...)     :', round(backoff_prob('transforms', ('machine',
'learning')), 5))
print('Interp P(improves|machine,learning):',
      round(deleted_interpolation('improves', 'machine', 'learning'), 3))

pred_dist = np.array([0.33, 0.33, 0.33]) # improves, enables, drives
print('Entropy of prediction distribution :', round(entropy(pred_dist), 3), 'bits')

best_word = ['improves', 'enables', 'drives'][int(np.argmax(pred_dist))]
print('Final next-word prediction      :', best_word)

```


Case Study 2: AI-Powered Hospital Appointment Chatbot

Given Data

- Sentence 1: “Book an appointment with the doctor.”
- Sentence 2: “The book contains medical information.”
- $P(\text{book} \mid \text{VB}) = 0.7$, $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$, $P(\text{Start} \rightarrow \text{NN}) = 0.4$

Q1. Assign Penn Treebank POS tags to every word in both sentences and justify the tag given to “book”.

Sentence 1: Book/VB an/DT appointment/NN with/IN the/DT doctor/NN ./.

Sentence 2: The/DT book/NN contains/VBZ medical/JJ information/NN ./.

Justification: In Sentence 1, “Book” opens the sentence with no preceding determiner and is followed directly by the noun phrase “an appointment” — the structure of an imperative request (“[You] book an appointment”), so it functions as a base-form verb (VB). In Sentence 2, “book” is preceded by the determiner “The” and followed by the verb “contains”, placing it in subject position as the head noun of the noun phrase “The book” — hence NN. The determiner immediately before the word is the strongest local cue that disambiguates the two readings.

Q2. Using the HMM probabilities, compute the likelihood of “book” as VB and as NN, and explain why the model favours one tag at the start of a sentence.

$$P(\text{VB}, \text{book}) = P(\text{Start} \rightarrow \text{VB}) \times P(\text{book} \mid \text{VB}) = 0.6 \times 0.7 = 0.42$$

$$P(\text{NN}, \text{book}) = P(\text{Start} \rightarrow \text{NN}) \times P(\text{book} \mid \text{NN}) = 0.4 \times 0.3 = 0.12$$

$$\text{Normalised: } P(\text{VB} \mid \text{book}) = 0.42 / (0.42 + 0.12) = 0.778 \quad P(\text{NN} \mid \text{book}) = 0.12 / 0.54 = 0.222$$

Since $0.42 > 0.12$, the HMM Viterbi decision selects VB as the most probable tag for “book” at the start of Sentence 1. The model favours VB here because both factors compound in the same direction: the transition probability from the sentence-start state is higher for VB (0.6 vs 0.4), and the emission (lexical) probability of “book” given VB is also higher (0.7 vs 0.3). Because the HMM tag decision is the product of these two probabilities rather than either one alone, VB dominates even though “book” is lexically ambiguous.

Q3. Compare Rule-Based POS Tagging and Stochastic HMM Tagging for resolving the ambiguity of “book”, and discuss their advantages/limitations in a healthcare chatbot.

Rule-Based Tagging

- Uses hand-written linguistic rules, e.g. “tag a word as VB if it is the first word of an imperative sentence”.
- Advantage: transparent, easy to audit, and precise for well-defined, frequent patterns — important where a hospital may need to explain/trace chatbot decisions.
- Limitation: does not scale — every new ambiguous pattern needs a new hand-crafted rule, and it struggles with sentence structures the rule-writer did not anticipate (e.g. informal patient phrasing).

Stochastic HMM Tagging

- Learns emission and transition probabilities automatically from a tagged training corpus.
- Advantage: generalises to novel sentences and phrasing automatically, which matters for a chatbot handling varied, informal patient input (“can i book a slot”, “i need to book an appt”, etc.).
- Limitation: requires a sufficiently large annotated corpus to estimate reliable probabilities, and its decisions are probabilistic/less directly explainable than a rule.

For a healthcare appointment chatbot, a hybrid approach is often best in practice: HMM tagging handles the bulk of everyday variation robustly, while a small set of high-precision rules can be layered on top for safety-critical disambiguation (e.g. always confirming “book” as an action before finalising an appointment).

Q4. Evaluate how the Penn Treebank tagset supports downstream tasks such as intent detection, entity identification, and response generation.

- Intent detection: A verb tag (VB/VBZ/VBP) reliably signals an action word (“book”, “cancel”, “reschedule”), letting the chatbot map the sentence to an intent class such as BOOK_APPOINTMENT without keyword-matching heuristics.
- Entity identification: Noun tags (NN/NNP/NNS) mark candidate entity spans — e.g. “doctor”, “appointment”, “Monday” — which downstream Named Entity Recognition can further classify as PERSON, SERVICE, or DATE.
- Response generation: Knowing the grammatical role of each token (subject, verb, object, modifier) lets a template- or grammar-based generator produce well-formed confirmation messages, and helps rank which slot (date, doctor, department) is still missing from the user's request.

Because the Penn Treebank tagset is a shared, standardised inventory of tags, models trained on outside corpora (news, dialogue, medical text) remain compatible with the chatbot's own pipeline, so pre-trained taggers and parsers can be reused directly rather than retrained from scratch.

Q5. Python program: tokenize the sentences, assign Penn Treebank POS tags, and implement the HMM calculation for “book” (VB vs NN).

```
import nltk
from nltk import word_tokenize, pos_tag

nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)

sentences = [
    "Book an appointment with the doctor.",
    "The book contains medical information."
]

print('--- Tokenisation & Penn Treebank POS Tagging ---')
for s in sentences:
    tokens = word_tokenize(s)
    tagged = pos_tag(tokens)
    print(f'Sentence : {s}')
    print(f'Tokens   : {tokens}')
    print(f'POS tags  : {tagged}')
    print()

# ---- HMM-style calculation for the ambiguous word 'book' ----
emission = {'VB': 0.7, 'NN': 0.3}      # P(book | tag)
transition = {'VB': 0.6, 'NN': 0.4}    # P(Start -> tag)

scores = {tag: emission[tag] * transition[tag] for tag in emission}
total = sum(scores.values())
normalised = {tag: round(score / total, 3) for tag, score in scores.items()}
best_tag = max(scores, key=scores.get)

print('--- HMM Probability Calculation for "book" ---')
for tag in scores:
    print(f'P({tag}, book) = P(Start->{tag}) x P(book|{tag}) = {scores[tag]:.3f}')
print('Normalised posterior probabilities:', normalised)
print('Most probable tag for "book" at sentence start ->', best_tag)
```


Case Study 3: Financial News POS Tag Correction and Uncertainty Analysis

Given Data

Sentence: “Market growth drives investment.”

- Initial tags: market/NN, growth/NN, drives/NNS, investment/NN
- Transformation rule: Change NNS → VBZ if the preceding word is tagged NN
- Corpus frequencies: market = 500, growth = 350, drives = 180, investment = 420

Q1. Apply the transformation rule to the initial POS-tag sequence, show the corrected sequence, and explain why the change is linguistically appropriate.

Initial sequence: market/NN → growth/NN → drives/NNS → investment/NN

The word immediately preceding “drives” is “growth”, tagged NN, so the rule “NNS → VBZ if preceding word is NN” fires on “drives”.

Corrected sequence: market/NN → growth/NN → drives/VBZ → investment/NN

This is linguistically appropriate because the sentence has a single noun-phrase subject “Market growth” (market modifying growth) followed by one main verb and one object “investment”. Read as a plural noun (NNS), “drives” would leave the sentence without a finite verb, which is ungrammatical. Read as VBZ (third-person singular present verb), “drives” correctly supplies the predicate: “[Market growth] drives [investment]”.

Q2. Analyze the initial tagging error for “drives” — why should it receive VBZ, and how does surrounding context reveal the correct tag?

The statistical tagger's initial NNS assignment likely reflects the surface form — “drives” ends in ‘-s’, a strong orthographic cue for plural nouns (e.g. “car drives”) — without checking sentence-level syntax. But the surrounding context shows this reading is wrong: “market growth” (NN + NN) forms a compact compound-noun subject with no article or additional noun, and “investment” (NN) directly follows “drives” as an object with no verb in between. A sentence needs exactly one finite verb, and the only candidate for that role is “drives”, which must therefore be the third-person-singular present-tense verb “to drive” (VBZ), meaning “causes” or “leads to”. This illustrates why transformation-based tagging, which looks at neighbouring tags rather than the word in isolation, catches errors that a purely lexical/statistical first pass misses.

Q3. Using the corpus statistics, calculate the relative frequency/probability of each word and explain how this supports probabilistic POS tagging.

Total occurrences = 500 + 350 + 180 + 420 = 1450

$$P(\text{market}) = 500 / 1450 \approx 0.345$$

$$P(\text{growth}) = 350 / 1450 \approx 0.241$$

$$P(\text{drives}) = 180 / 1450 \approx 0.124$$

$$P(\text{investment}) = 420 / 1450 \approx 0.290$$

These relative frequencies act as unigram priors: rarer words (like “drives” at 12.4%) contribute less-confident emission probabilities on their own, which is exactly why context-sensitive information (transition probabilities, transformation rules) is needed to disambiguate them. More broadly, corpus-derived frequencies are the empirical basis on which any probabilistic tagger (HMM, Brill's initial-state tagger, or a statistical baseline) estimates how likely a given tag is for a given word before context is even considered.

Q4. Calculate the entropy of the word-frequency distribution before and after applying the transformation, and interpret what entropy indicates about uncertainty.

(a) Entropy of the corpus word-frequency distribution (market, growth, drives, investment). The transformation rule only changes a POS tag, not how often each word occurs, so this distribution — and hence its entropy — is identical before and after the transformation:

$$\begin{aligned} H &= -\sum p(w) \log_2 p(w) \\ &= -[0.345 \times \log_2(0.345) + 0.241 \times \log_2(0.241) + 0.124 \times \log_2(0.124) + 0.290 \times \log_2(0.290)] \\ &\approx 0.530 + 0.495 + 0.374 + 0.518 \approx 1.92 \text{ bits (unchanged before/after)} \end{aligned}$$

(b) Entropy of the sentence's tag distribution, which does change with the correction. Before correction the four tags are {NN, NN, NNS, NN} → P(NN)=3/4, P(NNS)=1/4; after correction they are {NN, NN, VBZ, NN} → P(NN)=3/4, P(VBZ)=1/4 — the same 3:1 proportions, just a different minority label:

$$H_{\text{before}} = H_{\text{after}} = -[0.75 \times \log_2(0.75) + 0.25 \times \log_2(0.25)] \approx 0.311 + 0.500 = 0.811 \text{ bits}$$

Interpretation: entropy measures how uncertain/unpredictable a distribution is, not whether the labels are correct. Here it stays constant across the correction because entropy depends only on the shape of the probability distribution (3 common tags vs. 1 rare tag) — the transformation rule fixes which specific tag is wrong without changing how concentrated or spread out the tagging distribution is. A high entropy value elsewhere in a real corpus would flag sentences or words with genuinely uncertain/ambiguous tagging that deserve closer review (or more transformation rules), whereas low entropy indicates the tagger's decisions are concentrated and confident.

Q5. Python program: initial POS tagging, a simple Transformation-Based Tagger applying the given rule, corpus-based probabilities, and entropy calculation.

```
import nltk
import math
from nltk import word_tokenize

nltk.download('punkt', quiet=True)

sentence = "Market growth drives investment."
tokens = [w.lower() for w in word_tokenize(sentence) if w.isalpha()]

# ---- 1. Initial statistical POS tags (given/simulated) ----
initial_tags = ['NN', 'NN', 'NNS', 'NN']
print('Tokens      : ', tokens)
print('Initial tags : ', list(zip(tokens, initial_tags)))

# ---- 2. Transformation-Based Tagger: 'NNS -> VBZ if preceding tag is NN' ----
def apply_transformation(tokens, tags):
    corrected = tags.copy()
    for i in range(1, len(tags)):
        if tags[i] == 'NNS' and tags[i-1] == 'NN':
            corrected[i] = 'VBZ'
    return corrected

corrected_tags = apply_transformation(tokens, initial_tags)
print('Corrected tags:', list(zip(tokens, corrected_tags)))

# ---- 3. Corpus-based word probabilities ----
freq = {'market': 500, 'growth': 350, 'drives': 180, 'investment': 420}
total = sum(freq.values())
probs = {w: c / total for w, c in freq.items()}
print('Relative frequencies:', {w: round(p, 3) for w, p in probs.items()})
```

```

# ---- 4. Entropy calculations ----
def entropy(dist):
    return -sum(p * math.log2(p) for p in dist if p > 0)

word_entropy = entropy(probs.values())

tag_dist_before = [3/4, 1/4] # NN, NNS proportions
tag_dist_after  = [3/4, 1/4] # NN, VBZ proportions (same shape)

print('Entropy of word-frequency distribution :', round(word_entropy, 3), 'bits')
print('Entropy of tag distribution (before)   :', round(entropy(tag_dist_before), 3), 'bits')
print('Entropy of tag distribution (after)    :', round(entropy(tag_dist_after), 3), 'bits')

print('\nFinal POS-tagging result:', list(zip(tokens, corrected_tags)))

```

Integrated Syntactic Analysis Across Case Studies 1, 2 and 3

Although each case study targets a different NLP sub-system, all three rest on the same underlying idea: language is inherently ambiguous and sparse, and probabilistic models resolve that uncertainty using evidence from data rather than fixed rules alone.

- Case Study 1 (N-gram models) shows how word-level sequence prediction handles sparsity through backoff and interpolation, and how entropy quantifies how confident (or uncertain) a prediction really is — directly relevant to ranking autocomplete suggestions.
- Case Study 2 (HMM POS tagging) shows how the same probabilistic reasoning (combining a prior/transition probability with a lexical/emission probability) resolves word-level grammatical ambiguity, which is a prerequisite for correct intent detection in the chatbot.
- Case Study 3 (Brill tagging) shows that even a fast statistical first pass can be systematically wrong in predictable ways, and that lightweight, context-aware transformation rules — combined with corpus statistics and entropy-based confidence measures — can correct those errors cheaply.

Together, these techniques form a typical NLP pipeline: tokenisation → initial statistical tagging/prediction → context-aware correction/refinement → downstream application (search ranking, intent detection, analytics). Robust real-world systems, whether an e-commerce search bar, a hospital chatbot, or a financial news analytics engine, rely on exactly this combination of probabilistic language modelling, POS tagging, and uncertainty (entropy) analysis to make reliable decisions from ambiguous natural-language input.